Foundation models et génération

Jour 4 · Application au deep learning

ING3 option IA · CY Tech



Le cas d'étude de la séance

Pourquoi : toutes les notions du jour répondent à une même question, le matin comme au TD.

Se passer, en partie, des données annotées

- **Au TD 2** : 1 132 images annotées à la main, boîte par boîte, pour entraîner YOLO
- **Aujourd'hui** : des modèles déjà entraînés sur des centaines de millions d'images, sans annotation manuelle
- **Les données du TD** : les photos de chantier du TD 2, puis des images de vêtements

Les trois questions de la séance

1 — **Peut-on classer des images sans aucun exemple annoté ?** — parties 1 à 3

embeddings, foundation models, CLIP

2 — **Peut-on retrouver des images en les décrivant ?** — partie 4

un moteur de recherche par le texte

3 — **Comment générer des images, et choisir ce qu'elles montrent ?** — parties 5 à 7

la diffusion, puis la diffusion conditionnée

Source : TD 2 (Construction-PPE, 1 132 images d'entraînement annotées) ; Radford et al., CLIP, 2021.

Plan de la matinée

Pourquoi : le cours suit l'ordre du TD : chaque partie prépare une étape de l'après-midi.

9h30 · avant la pause

Partie 1 — 1
Embeddings et cosinus
rappel : comparer deux images
TD · étape 2

Partie 2 — 1
Les foundation models
un socle, puis une adaptation
TD · étape 2

Partie 3 — 1
CLIP
classer sans aucun exemple
TD · étape 2

Partie 4 — 2
La recherche d'images
chercher en décrivant
TD · étape 3

après la pause

Partie 5 — 3
Les modèles génératifs
VAE, GAN et diffusion
TD · étape 4

Partie 6 — 3
La diffusion
bruiter, puis débruiter
TD · étape 4

Partie 7 — 3
La diffusion conditionnée
choisir ce qu'on génère
TD · étape 5

Pour finir
La brique projet
les défauts rares
TD · étape 8

Supposé connu : le ViT (Jour 1), SAM (Jour 2), l'autoencodeur (Jour 3), la précision (Jour 2).



Partie 1

Embeddings et similarité cosinus

Un rappel : comment comparer deux images par leur contenu

Embeddings et similarité cosinus : définition

Définition

Un embedding est un vecteur qui résume une image (ou un texte) : deux images qui se ressemblent ont des vecteurs proches. La similarité cosinus mesure cette proximité : le cosinus de l'angle entre les deux vecteurs.

Le problème que cela résout

Comparer deux images pixel par pixel ne dit rien de leur contenu. Comparer leurs embeddings, si : c'est la base de la classification et de la recherche d'images du jour.

Ce qui entre, ce qui sort

Entrée : deux vecteurs, par exemple deux embeddings de 512 nombres. Sortie : un nombre entre −1 et 1 ; 1 s'ils pointent dans la même direction.

Ce que l'on va voir, dans l'ordre

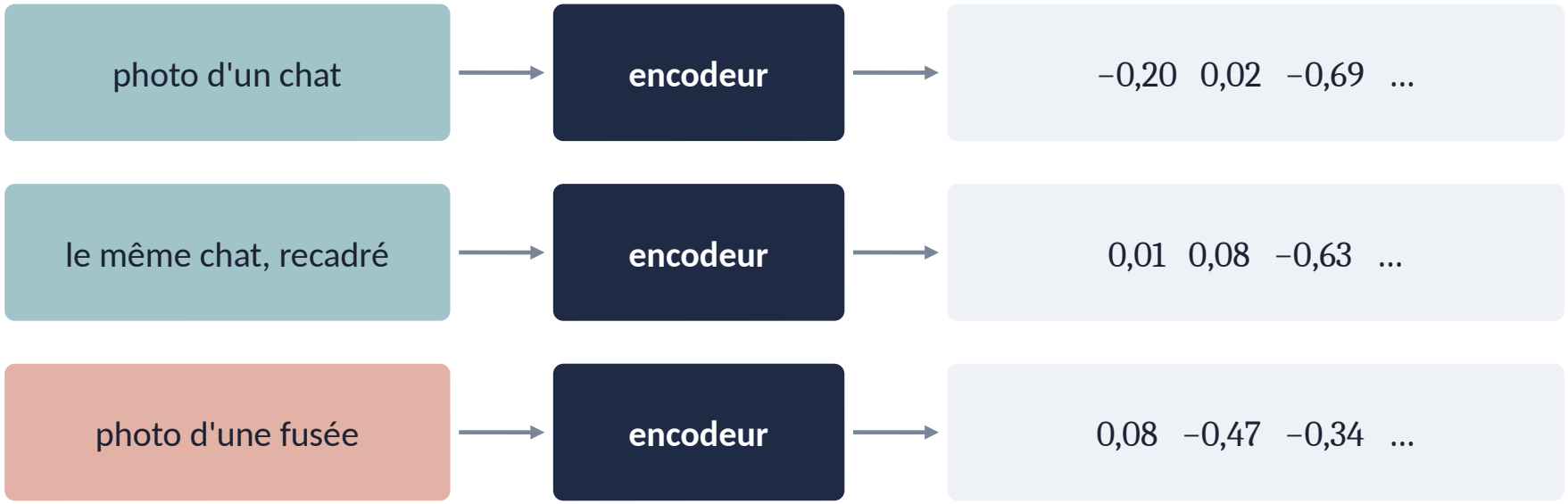
2. Un embedding résume une image
3. La similarité cosinus
4. La similarité cosinus en code
5. À retenir

Dans le TD : Étape 2 : TODO 1 (la similarité cosinus entre deux ensembles de vecteurs).

Un embedding résume une image

Pourquoi : c'est l'objet que manipulent CLIP, DINO et la recherche d'images.

Un réseau transforme une image en un **vecteur**, son **embedding**. Deux images au contenu proche donnent des vecteurs proches.



Les 3 premiers des 512 nombres, calculés avec le CLIP du TD (photos d'exemple de scikit-image).

0,98

similarité entre le chat et le même chat recadré

0,28

entre le chat et la fusée (cosinus : diapositive suivante)

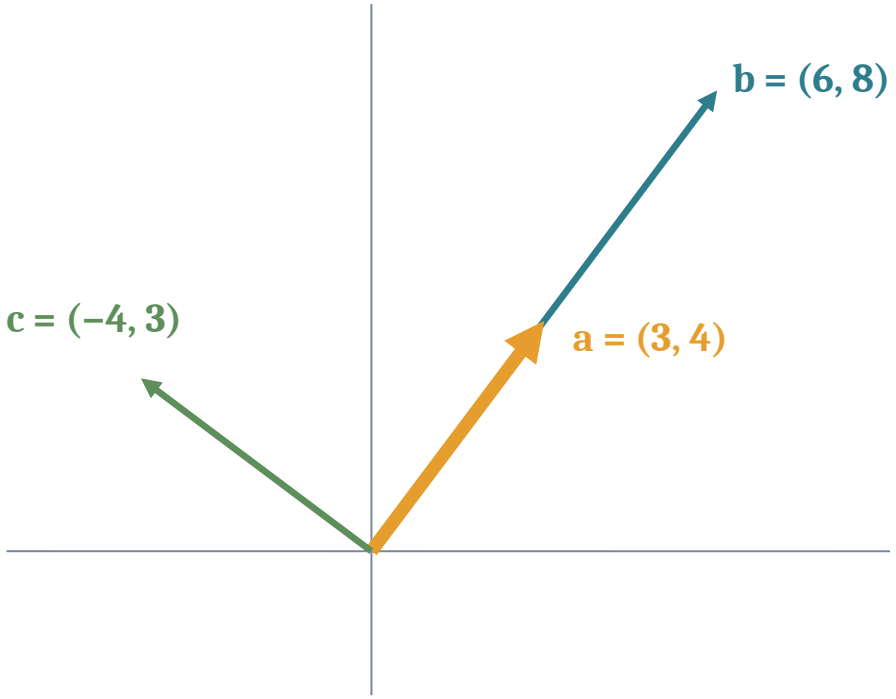
Les nombres ne se lisent pas un par un : ce qui compte, c'est la **proximité** entre vecteurs.

Source : CLIP du TD (OpenCLIP ViT-B/32) sur des photos d'exemple de scikit-image ; calculé pour le cours.

La similarité cosinus

Pourquoi : la mesure de ressemblance de toute la journée, celle du TODO 1.

On mesure la proximité de deux vecteurs par le **cosinus de l'angle** entre eux : **1** même direction, **0** perpendiculaires, **−1** opposés.



	cosinus	pourquoi
a et b	1	même direction (b = 2 × a)
a et c	0	perpendiculaires
a et −a	−1	directions opposées

$$\cos(a, b) = (a \cdot b) \,/\, (\|a\| \times \|b\|)$$

La longueur ne compte pas : on divise chaque vecteur par sa norme, puis on fait un produit scalaire.

La similarité cosinus en code

Pourquoi : vérifier les trois cas sur des vecteurs que l'on calcule de tête.

```python
import numpy as np

def cosinus(u, v):
    return u @ v / (np.linalg.norm(u) * np.linalg.norm(v))

a = np.array([3., 4.])
b = np.array([6., 8.])       # même direction que a, deux fois plus long
c = np.array([-4., 3.])      # perpendiculaire à a

print(cosinus(a, b))         # 1.0 : même direction
print(cosinus(a, c))         # 0.0 : perpendiculaires
print(cosinus(a, -a))        # -1.0 : directions opposées
print(np.linalg.norm(a))     # 5.0 : la longueur ne compte pas
```

- Produit scalaire divisé par le produit des **normes**
- **b** est deux fois plus long que **a** : le cosinus vaut quand même 1
- Au TODO 1 : la même chose entre **deux ensembles** de vecteurs, avec un produit matriciel

Tous les extraits de code du cours ont été exécutés avec les versions de uv.lock (PyTorch 2.14.1, NumPy) : les sorties écrites en commentaire sont celles obtenues.

Source : Documentation NumPy : numpy.linalg.norm.

Embeddings et similarité cosinus : à retenir

1. Un **embedding** résume une image en un vecteur ; au TD, **512 nombres** produits par CLIP.

2. La **similarité cosinus** vaut 1 pour deux vecteurs de même direction, 0 s'ils sont perpendiculaires, −1 s'ils sont opposés.

3. Seule la **direction** compte, pas la longueur : on divise chaque vecteur par sa norme, puis on fait un produit scalaire.

Pour approfondir : d2l, section 2.3 : Linear Algebra (produit scalaire et norme)



Partie 2

Les foundation models

Un grand modèle entraîné une fois, réutilisé pour de nombreuses tâches

Les foundation models : définition

Définition

Un foundation model est un grand modèle entraîné une fois, à très grande échelle, sur des données pas ou peu annotées. Ses représentations servent ensuite à de nombreuses tâches, avec une adaptation légère : une phrase, quelques exemples, ou un court réentraînement.

Le problème que cela résout

Annoter coûte cher : au TD 2, il a fallu 1 132 images annotées à la main pour entraîner YOLO. Un foundation model permet souvent de démarrer avec très peu d'annotations, voire aucune.

Ce qui entre, ce qui sort

Entrée : un modèle déjà entraîné (CLIP, DINO, SAM). Sortie : une solution à votre problème, après une adaptation légère.

Ce que l'on va voir, dans l'ordre

2. Un socle, puis une adaptation
3. Apprendre sans annotation manuelle
4. Trois façons d'adapter un socle
5. À retenir

Dans le TD : Étape 2 : zéro-shot et sonde linéaire ; étape 7 : le bilan, foundation model ou modèle spécialisé.

Un socle, puis une adaptation

Pourquoi : ce qui change quand on part d'un foundation model.

On passe d'**un modèle par problème** à **un socle entraîné une fois**, puis **adapté** à chaque problème.

Avant

annoter — des milliers d'images, à la main

↓

entraîner — un modèle pour ce problème

↓

recommencer — pour le problème suivant

Avec un foundation model

le socle — entraîné une fois, par d'autres, à très grande échelle

↓

adapter — une phrase, quelques exemples, ou un court réentraînement

↓

réutiliser — le même socle pour d'autres problèmes

Au TD 2 : **1 132 images annotées** à la main pour un seul problème. Aujourd'hui, on part d'un modèle qui n'a jamais vu ces images.

Apprendre sans annotation manuelle

Pourquoi : comprendre d'où viennent les connaissances de ces modèles.

L'annotation manuelle est remplacée par une **astuce** : des légendes trouvées sur le web, deux vues d'une même image, ou des masques produits en masse.

Modèle	Comment il apprend	Ce qu'il permet	Au TD
CLIP 2021	rapproche chaque image de sa légende, parmi 400 millions de paires trouvées sur le web	classer et chercher des images par le texte	étapes 2 et 3
DINO 2021, DINOv2 en 2023	deux vues différentes de la même image doivent donner le même embedding ; aucun texte	chercher des images qui se ressemblent	étape 8
SAM 2023	plus d'un milliard de masques, produits en grande partie automatiquement par le modèle lui-même	segmenter n'importe quel objet (Jour 2)	TD 2



Tous trois utilisent un **Vision Transformer** (le ViT du Jour 1) pour encoder les images.

Source : Radford et al., CLIP, 2021 ; Caron et al., DINO, 2021 ; Oquab et al., DINOv2, 2023 ; Kirillov et al., SAM, 2023.

Trois façons d'adapter un socle

Pourquoi : les trois options que le TD compare à l'étape 2.

De la moins à la plus coûteuse : décrire les classes par des **phrases**, entraîner une **petite couche** sur quelques exemples, ou **réentraîner** le modèle.

Zéro-shot

0 image annotée

une phrase par classe ; l'image prend la classe la plus proche

étape 2, TODO 2

Sonde linéaire

quelques images par classe

une régression logistique sur les embeddings ; le modèle reste figé

étape 2

Fine-tuning

des centaines d'images ou plus

on réentraîne tout ou partie du modèle (Jour 1)

Jour 1

de plus en plus d'images annotées et de calcul



Au TD, ces approches sont comparées à une baseline classique (descripteur fait main et SVM), avec le même nombre d'exemples.

Source : Radford et al., CLIP, 2021, section 3 (zéro-shot et sonde linéaire).

Les foundation models : à retenir

1. On passe d'**un modèle par problème** à **un socle, puis une adaptation**.

2. Le socle apprend **sans annotation manuelle** : des légendes trouvées sur le web (CLIP), deux vues d'une même image (DINO), des masques produits automatiquement en masse (SAM).

3. Trois façons d'adapter, de la moins à la plus coûteuse : **zéro-shot**, **sonde linéaire**, **fine-tuning**.

Pour approfondir : Bommasani et al., 2021 : On the Opportunities and Risks of Foundation Models (arXiv)



Partie 3

CLIP, des images et des textes

Le foundation model du TD : classer une image sans aucun exemple

CLIP : définition

Définition

CLIP est un foundation model qui relie images et textes. Deux encodeurs, l'un pour les images, l'autre pour les textes, projettent dans un même espace ; il a appris à rapprocher chaque image de sa légende, sur des centaines de millions de paires du web.

Le problème que cela résout

Pour classer, il n'a besoin d'aucune image annotée : on écrit une phrase par classe, et l'image prend la classe de la phrase la plus proche. C'est la classification zéro-shot.

Ce qui entre, ce qui sort

Entrée : une image et une liste de phrases. Sortie : une probabilité par phrase.

Ce que l'on va voir, dans l'ordre

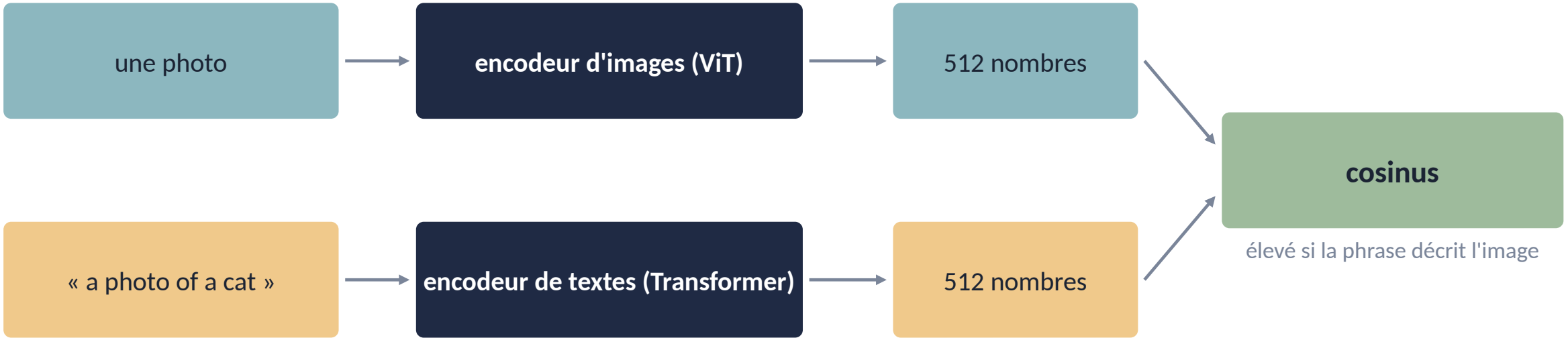
2. Deux encodeurs, un même espace
3. L'apprentissage contrastif
4. Classer sans exemple : le zéro-shot
5. La température
6. Les phrases comptent
7. Le zéro-shot en code
8. À retenir

Dans le TD : Étape 2 : TODO 2 (le zéro-shot), puis le choix des phrases.

Deux encodeurs, un même espace

Pourquoi : l'architecture de CLIP, utilisée tout le TD.

Un encodeur pour les **images**, un pour les **textes** : tous deux produisent 512 nombres **dans le même espace**. On peut donc comparer une image et une phrase.



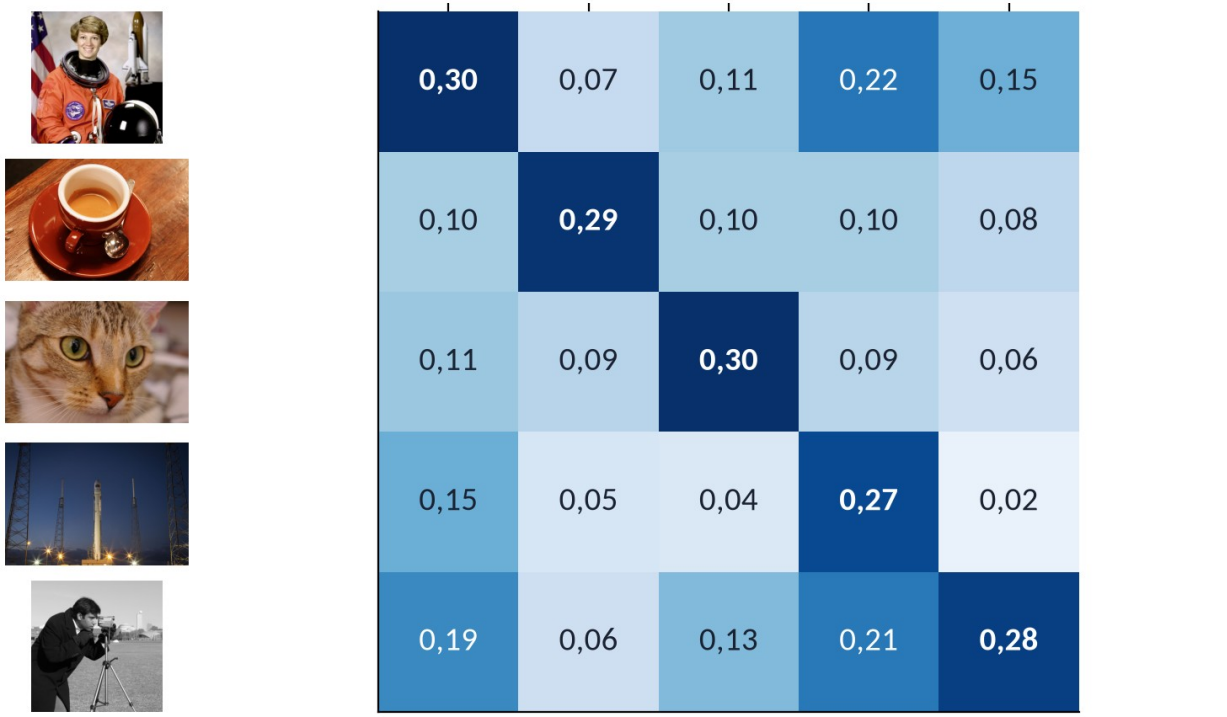
Au TD : **ViT-B/32** d'OpenCLIP, entraîné sur **400 millions** de paires image-texte du web (LAION-400M).

Source : Radford et al., CLIP, 2021 ; poids du TD : OpenCLIP ViT-B/32 entraîné sur LAION-400M.

L'apprentissage contrastif

Pourquoi : comment CLIP apprend à rapprocher une image de sa légende.

Dans un lot de paires image-légende, chaque image doit être **plus proche de sa légende** (la diagonale) **que de toutes les autres**.



	an astronaut	a cup of coffee	a cat	a rocket	a man with a camera
	0,30	0,07	0,11	0,22	0,15
	0,10	**0,29**	0,10	0,10	0,08
	0,11	0,09	**0,30**	0,09	0,06
	0,15	0,05	0,04	**0,27**	0,02
	0,19	0,06	0,13	0,21	**0,28**

- Le tableau : le **cosinus** de chaque photo avec chaque phrase, calculé par le CLIP du TD
- **Sur la diagonale**, les bonnes paires : de 0,27 à 0,30
- **Ailleurs** : 0,22 au plus
- L'entraînement pousse la diagonale **vers le haut**, le reste **vers le bas**

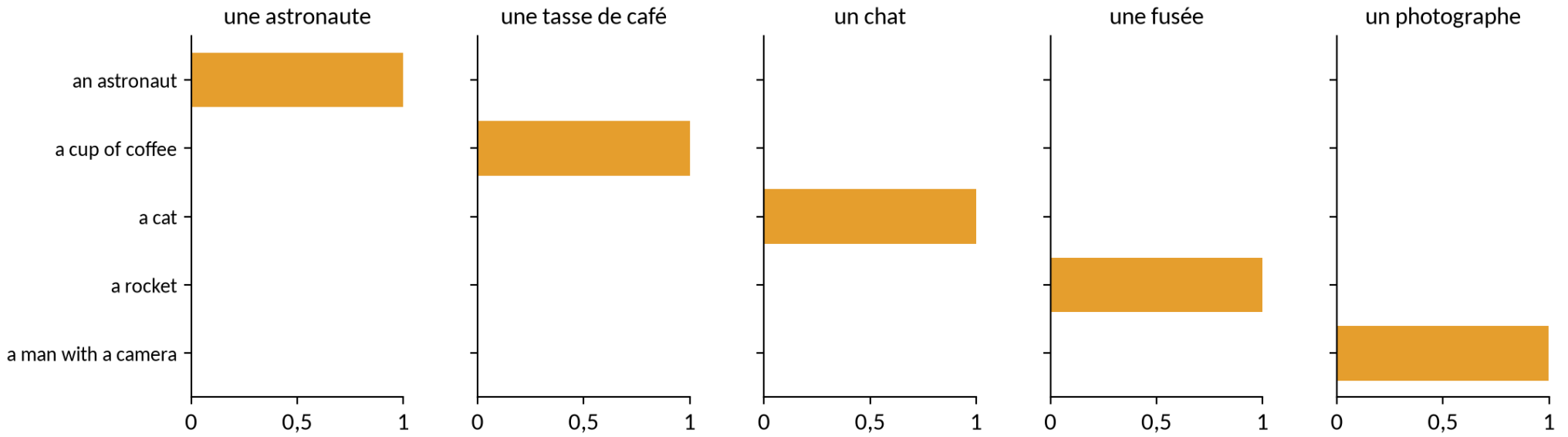
Source : CLIP du TD (OpenCLIP ViT-B/32, LAION-400M) sur cinq photos d'exemple de scikit-image ; calculé pour le cours.

Classer sans exemple : le zéro-shot

Pourquoi : la classification du TODO 2, sans aucune image annotée.

Une **phrase par classe** ; l'image prend la classe de la **phrase la plus proche**. Aucune image annotée : c'est le **zéro-shot**.

écrire une phrase par classe → encoder l'image et les phrases → cosinus × 100, puis softmax → **la classe la plus probable**



Les cinq photos de la diapositive précédente : la bonne classe l'emporte chaque fois (probabilité d'au moins 0,998). Ces photos sont faciles ; au TD, on mesure sur les vignettes de chantier.

Source : CLIP du TD sur cinq photos d'exemple de scikit-image ; calculé pour le cours.

La température

Pourquoi : le réglage qui rend la décision de CLIP tranchée.

Les cosinus de CLIP sont **petits et proches** : on les multiplie par **100** avant le softmax, pour que la décision soit **tranchée**.

Deux classes, cosinus 0,25 et 0,30

Sans température

softmax(0,25 ; 0,30)

0,49 et 0,51

le modèle semble hésiter

Avec la température 100

softmax(25 ; 30)

0,007 et 0,993

la décision est tranchée

La classe choisie **ne change pas**, seule la confiance affichée change. Dans CLIP, la température est **apprise** et plafonnée à 100. C'est le TODO 2.

Source : Radford et al., CLIP, 2021 (température apprise, plafonnée à 100) ; calculé pour le cours.

Les phrases comptent

Pourquoi : écrire les classes, c'est régler le système : le TD vous le fait mesurer.

Même modèle, même image : **changer les phrases change le résultat**. Écrire les classes, c'est **régler** le système.

La photo du chat, deux listes de trois phrases

Liste des phrases (« a photo of a … »)	Probabilités
cat · dog · rocket	1,00 · 0,00 · 0,00
cat · tabby cat · kitten	0,20 · 0,79 · 0,01

Avec la deuxième liste, « cat » ne reçoit plus que 0,20 : les trois classes se recouvrent et se partagent la probabilité. Le softmax compare les phrases **entre elles** : elles doivent bien se distinguer.

Au TD : choisir les phrases sur **une moitié** des vignettes, mesurer sur **l'autre**, comme validation et test. Plusieurs formulations à essayer.

Source : CLIP du TD sur la photo du chat de scikit-image ; calculé pour le cours. Radford et al., 2021, section 3.1.4 (prompt engineering).

Le zéro-shot en code

Pourquoi : le zéro-shot complet, du chargement du modèle à la décision.

```python
import open_clip
import torch
from PIL import Image
from skimage.data import chelsea

modele, _, pretraitement = open_clip.create_model_and_transforms(
    "ViT-B-32-quickgelu", pretrained="modeles/clip_vitb32_laion400m.pt")
tokeniseur = open_clip.get_tokenizer("ViT-B-32-quickgelu")
phrases = ["a photo of a cat", "a photo of a dog", "a photo of a rocket"]

with torch.no_grad():
    image = modele.encode_image(pretraitement(Image.fromarray(chelsea()))[None])
    textes = modele.encode_text(tokeniseur(phrases))
image = image / image.norm(dim=-1, keepdim=True)
textes = textes / textes.norm(dim=-1, keepdim=True)

cos = image @ textes.T                              # cosinus image / phrase
print([round(v, 3) for v in cos[0].tolist()])       # [0.299, 0.164, 0.086]
proba = (100 * cos).softmax(dim=-1)                 # température 100, puis softmax
print([round(v, 3) for v in proba[0].tolist()])     # [1.0, 0.0, 0.0]
```

- Le modèle et ses poids : ceux du TD
- **encode_image** et **encode_text** : 512 nombres chacun
- On **normalise**, puis un produit donne les cosinus
- Température **100**, puis **softmax** : la photo du chat est classée « cat »

Source : Documentation OpenCLIP.

CLIP : à retenir

1. CLIP projette **images et textes dans un même espace** ; il a appris en rapprochant chaque image de **sa** légende, et en l'éloignant des autres.

2. **Zéro-shot** : une phrase par classe, cosinus × **température** (100), puis softmax ; l'image prend la classe la plus probable.

3. Les **phrases** font partie du réglage : on les choisit sur une partie des images, on mesure sur une autre.

Pour approfondir : Radford et al., 2021 : l'article CLIP (arXiv) ; OpenCLIP, la bibliothèque utilisée au TD



Partie 4

La recherche d'images

Retrouver des images en les décrivant, sans aucun mot-clé

La recherche d'images : définition

Définition

Un moteur de recherche d'images par le texte encode une fois toutes les images d'un corpus, puis, pour chaque requête écrite, renvoie les images dont l'embedding est le plus proche de celui de la requête.

Le problème que cela résout

Retrouver « un ouvrier sans casque » dans des milliers de photos, sans aucun mot-clé associé aux images, et sans entraîner de modèle.

Ce qui entre, ce qui sort

Entrée : une requête en texte (ou une image) et un corpus d'images. Sortie : les k images les plus proches, de la plus proche à la moins proche.

Ce que l'on va voir, dans l'ordre

2. Chercher une image en la décrivant
3. La précision à k
4. Chercher par l'image : CLIP face aux histogrammes
5. À retenir

Dans le TD : Étape 3 : TODO 3 (la recherche) et TODO 4 (la précision à k).

Chercher une image en la décrivant

Pourquoi : le moteur de recherche de l'étape 3, TODO 3.

On encode le corpus **une fois** ; chaque requête écrite est comparée à **toutes** les images, puis on **trie** les cosinus.

une fois

encoder les 500 photos du corpus

500 × 512 nombres

pour chaque requête

encoder la phrase → **cosinus avec les 500 images** → **trier, garder les k premières**

« a worker on a roof » — un produit matriciel — les résultats

Aucun mot-clé n'est associé aux images : certaines notions cherchées au TD n'ont jamais été annotées. C'est le TODO 3.

La précision à k

Pourquoi : évaluer un moteur de recherche : le TODO 4.

Précision à k : parmi les **k premiers** résultats, la **part de résultats pertinents**.



les 5 premiers résultats d'une requête, dans l'ordre

3 / 5 = 0,6

précision à 5

2 / 2 = 1

précision à 2

La précision à k ne voit pas les images pertinentes **manquées** : c'est le rôle du **rappel** (Jour 2). C'est le TODO 4.

Chercher par l'image : CLIP face aux histogrammes

Pourquoi : la baseline classique de la recherche d'images.

Avec une **image** comme requête : la baseline classique compare des **couleurs**, CLIP compare des **embeddings appris**.

Histogrammes de couleur

la baseline classique

- combien de pixels de chaque couleur dans l'image
- deux images sont proches si leurs couleurs se répartissent de la même façon
- aucun apprentissage

Embeddings CLIP

le foundation model

- 512 nombres appris sur 400 millions de paires
- deux images sont proches si leurs embeddings sont proches (similarité cosinus)
- aucun entraînement de votre part



Au TD : les deux méthodes sur les **mêmes images requêtes**, à comparer par vous-mêmes.

Source : Swain et Ballard, Color Indexing, IJCV 1991 (histogrammes de couleur).

La recherche d'images : à retenir

1. On encode le corpus **une fois** ; chaque requête ne coûte ensuite qu'un **produit matriciel** et un tri.

2. La **précision à k** : la part de résultats pertinents parmi les k premiers. « Ça a l'air de marcher » n'est pas une évaluation.

3. La baseline classique compare des **histogrammes de couleur** ; CLIP compare des **embeddings appris**. Le TD compare les deux.

Pour approfondir : Radford et al., 2021 : l'article CLIP (arXiv)



Partie 5

Les modèles génératifs

Créer des images qui n'existaient pas

Les modèles génératifs : définition

Définition

Un modèle génératif apprend à quoi ressemblent les données d'entraînement, puis en tire au sort de nouvelles : des images qui n'existaient pas, mais qui ressemblent aux vraies. Trois grandes familles : VAE, GAN et diffusion.

Le problème que cela résout

Générer des images réalistes et variées, par exemple pour compléter une classe dont on n'a que quelques exemples.

Ce qui entre, ce qui sort

Entrée : un tirage au hasard (du bruit), et éventuellement une condition (une classe, une phrase). Sortie : une nouvelle image.

Ce que l'on va voir, dans l'ordre

2. Générer, c'est tirer au sort de nouvelles données
3. VAE, GAN et diffusion
4. À retenir

Dans le TD : Étapes 4 et 5 : la diffusion, d'abord sur des points du plan, puis sur des images.

Générer, c'est tirer au sort de nouvelles données

Pourquoi : ce qu'on attend d'un modèle génératif.

Un modèle génératif apprend **à quoi ressemblent** les données, puis en **tire au sort de nouvelles** : réalistes, variées, et pas des copies.

1 · apprendre — **données d'entraînement** → **modèle génératif** — par exemple 60 000 images de vêtements (TD, étape 5)

2 · générer — **un tirage au hasard** → **modèle génératif** → **une image nouvelle** — un tirage différent donne une image différente

Réaliste

on ne la distingue pas d'une vraie image

Variée

pas toujours la même image

Nouvelle

pas une copie d'une image d'entraînement

Utilité industrielle : compléter une **classe rare** (volet défauts rares du projet), produire des images d'essai.

VAE, GAN et diffusion

Pourquoi : situer les trois grandes familles avant de détailler la diffusion.

Trois familles pour **générer** ; la **diffusion** est l'approche des générateurs d'images actuels.

VAE

2013

un autoencodeur (Jour 3) dont on **tire le code au hasard**, puis on le décode

\+ simple et stable

− images souvent floues

GAN

2014

un **générateur** fabrique, un **juge** distingue le vrai du faux ; chacun progresse contre l'autre

\+ images nettes

− entraînement instable

Diffusion

2020

on apprend à **retirer du bruit**, petit à petit, à partir d'un bruit pur

\+ stable, qualité élevée

− génération lente : de nombreuses étapes

Source : Kingma et Welling, 2013 (VAE) ; Goodfellow et al., 2014 (GAN) ; Ho, Jain et Abbeel, 2020 (diffusion, DDPM).

Les modèles génératifs : à retenir

1. Générer, c'est **tirer au sort** de nouvelles données qui ressemblent aux données d'entraînement, sans les recopier.

2. **VAE** : un autoencodeur dont on peut tirer le code au hasard ; images souvent floues. **GAN** : un générateur contre un juge ; images nettes, entraînement instable.

3. **Diffusion** : on apprend à retirer du bruit ; entraînement stable, qualité élevée : c'est l'approche dominante aujourd'hui.

Pour approfondir : Goodfellow, Bengio et Courville, Deep Learning, chapitre 20 : Deep Generative Models



Partie 6

La diffusion : bruiter, puis débruiter

Apprendre à retirer du bruit, puis créer une image à partir d'un bruit pur

La diffusion : définition

Définition

Un modèle de diffusion apprend à retirer du bruit. On abîme les données en leur ajoutant du bruit en de nombreuses petites étapes ; un réseau apprend à retrouver ce bruit. Pour générer, on part d'un bruit pur et l'on débruite étape par étape.

Le problème que cela résout

Abîmer une image est facile, la créer est difficile. La diffusion découpe la création en 200 petites corrections, chacune facile à apprendre.

Ce qui entre, ce qui sort

Entrée : une image bruitée et le numéro de l'étape. Sortie (du réseau) : le bruit qu'elle contient. Sortie (de la génération) : une image.

Ce que l'on va voir, dans l'ordre

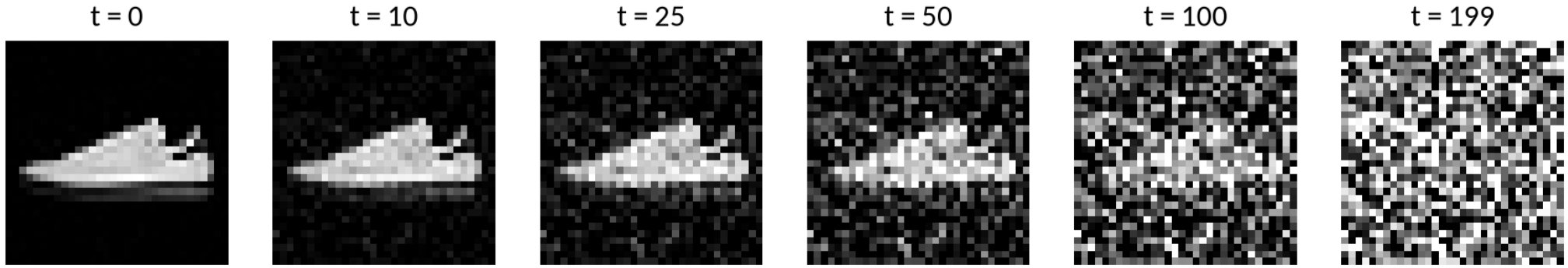
2. Bruiter : détruire l'image pas à pas
3. Sauter directement à l'étape t
4. Apprendre à retrouver le bruit
5. Générer : débruiter pas à pas
6. Le planning de bruit en code
7. À retenir

Dans le TD : Étape 4 : TODO 5 (bruiter) et TODO 6 (une étape de débruitage), sur des points du plan.

Bruiter : détruire l'image pas à pas

Pourquoi : le processus que le modèle apprendra à inverser.

On abîme une image en lui ajoutant du bruit, **en 200 petites étapes**. Le modèle apprendra à faire **le chemin inverse**.



bruiter : facile, aucun apprentissage

débruiter : ce que le réseau apprend

étape t	0	50	100	199
part de l'image	1,00	0,88	0,60	0,13
part du bruit	0,01	0,48	0,80	0,99

Planning du TD : 200 étapes, calculé pour le cours (le code est en fin de partie).

Source : Fashion-MNIST (Xiao et al., 2017) et planning de bruit du TD (200 étapes) ; calculé pour le cours.

Sauter directement à l'étape t

Pourquoi : la formule du TODO 5.

Pas besoin d'ajouter le bruit 50 fois : on **saute directement** à l'étape t en **mélangeant** l'image et un bruit, avec deux coefficients qui ne dépendent que de t.

image bruitée à l'étape t = (part de l'image) × image + (part du bruit) × bruit

t = 50	0,88 × image + 0,48 × bruit
t = 199	0,13 × image + 0,99 × bruit

Le bruit est tiré au hasard (loi normale), d'une forme identique à l'image. Les deux parts viennent du planning de bruit du TD.

C'est le **TODO 5** : la même formule pour des points du plan et pour des images.

Pour les curieux : $x_t = \sqrt{\bar{\alpha}_t} \times x_0 + \sqrt{(1 - \bar{\alpha}_t)} \times \varepsilon$, avec ε tiré selon une loi normale ; $\bar{\alpha}_t$ (alphas_cumules au TD) décroît de 1 vers 0.

Source : Ho, Jain et Abbeel, Denoising Diffusion Probabilistic Models, NeurIPS 2020, équation 4.

Apprendre à retrouver le bruit

Pourquoi : ce que le réseau apprend vraiment pendant l'entraînement.

Le réseau voit une donnée **bruitée** et l'étape t ; il doit **retrouver le bruit** qu'on a ajouté.

1	2	3	4
prendre une donnée	**tirer une étape t**	**tirer un bruit et bruiter**	**retrouver le bruit**
un point, ou une image	au hasard, entre 0 et 199	avec la formule du TODO 5	le réseau voit la donnée bruitée et t

$$\text{perte} = \text{moyenne de } (\text{bruit prédit} - \text{bruit vrai})^2$$

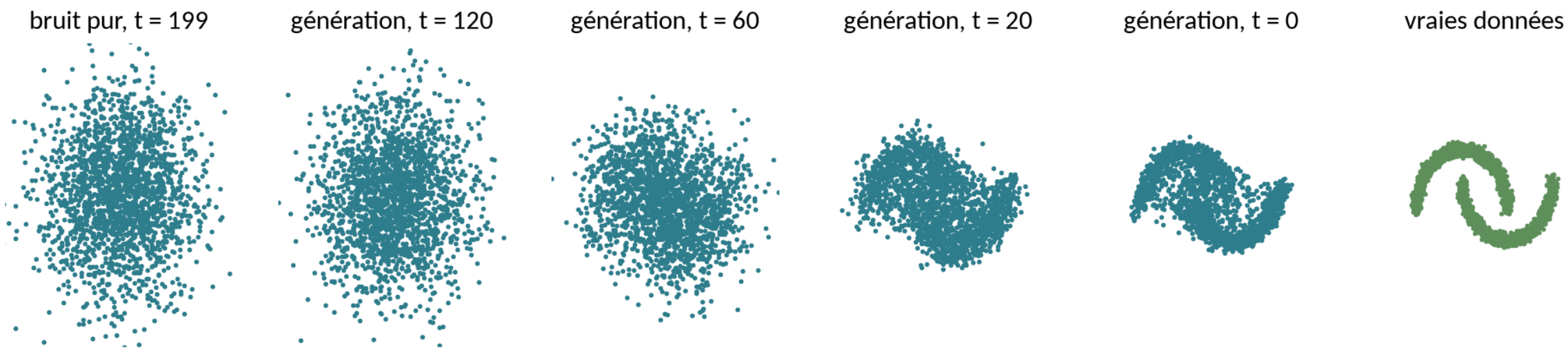
Réseau du TD : un **petit réseau dense** pour les points du plan, un **U-Net** (TD 2) pour les images. L'étape t lui est donnée, encodée par des sinus et cosinus.

Source : Ho, Jain et Abbeel, 2020, algorithme 1.

Générer : débruiter pas à pas

Pourquoi : la génération, avec l'étape du TODO 6.

On part d'un **bruit pur** ; 200 fois : le réseau **prédit le bruit**, on en **retire une partie**, on rajoute un **petit bruit frais** (sauf à la dernière étape).



Calculé pour le cours : le débruiteur du TD, entraîné 3 000 itérations sur « deux lunes », génère 2 000 points.

Une étape de débruitage, avec son **bruit frais** (sauf à la dernière étape) : c'est le **TODO 6**. À quoi sert ce bruit frais ? Une question du TD.

Source : Ho, Jain et Abbeel, 2020, algorithme 2 ; débruiteur du TD entraîné 3 000 itérations sur « deux lunes », calculé pour le cours.

Le planning de bruit en code

Pourquoi : lire les deux coefficients de la formule à chaque étape.

```python
import torch

betas = torch.linspace(1e-4, 0.04, 200)         # bruit ajouté à chaque étape
alphas_cumules = torch.cumprod(1 - betas, dim=0)

for t in (0, 50, 100, 199):
    signal = alphas_cumules[t].sqrt().item()
    bruit = (1 - alphas_cumules[t]).sqrt().item()
    print(t, round(signal, 2), round(bruit, 2))
# 0 1.0 0.01
# 50 0.88 0.48
# 100 0.6 0.8
# 199 0.13 0.99
```

- **betas** : le bruit ajouté à chaque étape, de plus en plus fort
- **alphas_cumules** : le produit des (1 − beta) jusqu'à t
- Part de l'image : sa **racine** ; part du bruit : la racine de **1 − alphas_cumules**
- Ce sont les nombres du tableau de la partie

Source : Documentation PyTorch : torch.cumprod.

La diffusion : à retenir

1. **Bruiter** : on mélange l'image et un bruit ; la part de l'image diminue d'étape en étape. On saute directement à l'étape t avec une formule.

2. **Entraîner** : on montre au réseau une image bruitée et l'étape t ; il doit **retrouver le bruit**. La perte est une erreur quadratique.

3. **Générer** : on part d'un bruit pur et l'on applique 200 fois l'étape de débruitage ; un petit bruit frais est rajouté, sauf à la dernière étape.

Pour approfondir : Ho, Jain et Abbeel, 2020 : l'article DDPM (arXiv) ; L. Weng, What are Diffusion Models?



Partie 7

La diffusion conditionnée

Choisir ce que l'on génère : une classe, ou une phrase

La diffusion conditionnée : définition

Définition

Un modèle de diffusion conditionné reçoit, en plus de l'image bruitée, une indication de ce qu'il doit générer : une classe, ou une phrase. Un réglage, la force de la condition, dit à quel point il doit la suivre.

Le problème que cela résout

Un modèle non conditionné génère n'importe quelle image vraisemblable. On veut choisir : « un pantalon », ou « un casque de chantier bleu ».

Ce qui entre, ce qui sort

Entrée : un bruit pur, une condition, une force g. Sortie : une image de la classe ou de la description demandée.

Ce que l'on va voir, dans l'ordre

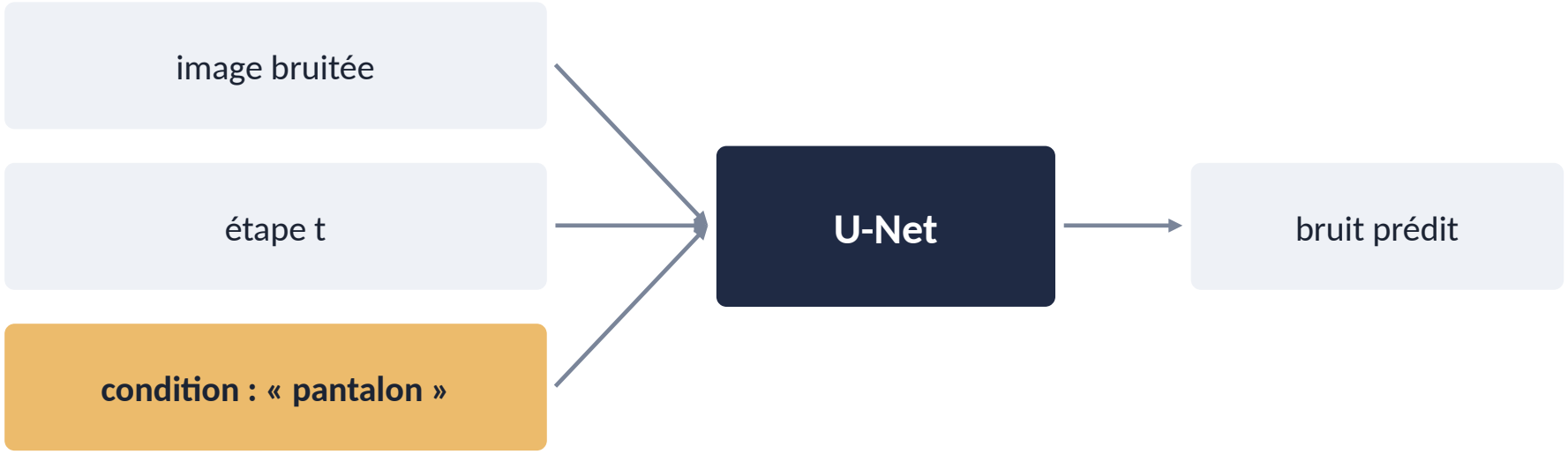
2. Ajouter une condition
3. La force de la condition
4. Stable Diffusion : la diffusion dans un espace compressé
5. À retenir

Dans le TD : Étape 5 : générer des vêtements avec le U-Net fourni, en faisant varier la force de la condition.

Ajouter une condition

Pourquoi : choisir ce que l'on génère, comme le texte dans Stable Diffusion.

Le réseau reçoit **une information de plus** : la **condition**. Une classe au TD ; une **phrase encodée par CLIP** dans Stable Diffusion.



Stable Diffusion

la condition est une phrase, passée dans l'encodeur de textes d'un CLIP (plus grand que celui du TD)

À l'entraînement, on **cache parfois la condition** : le réseau apprend à prédire le bruit **avec et sans** elle.

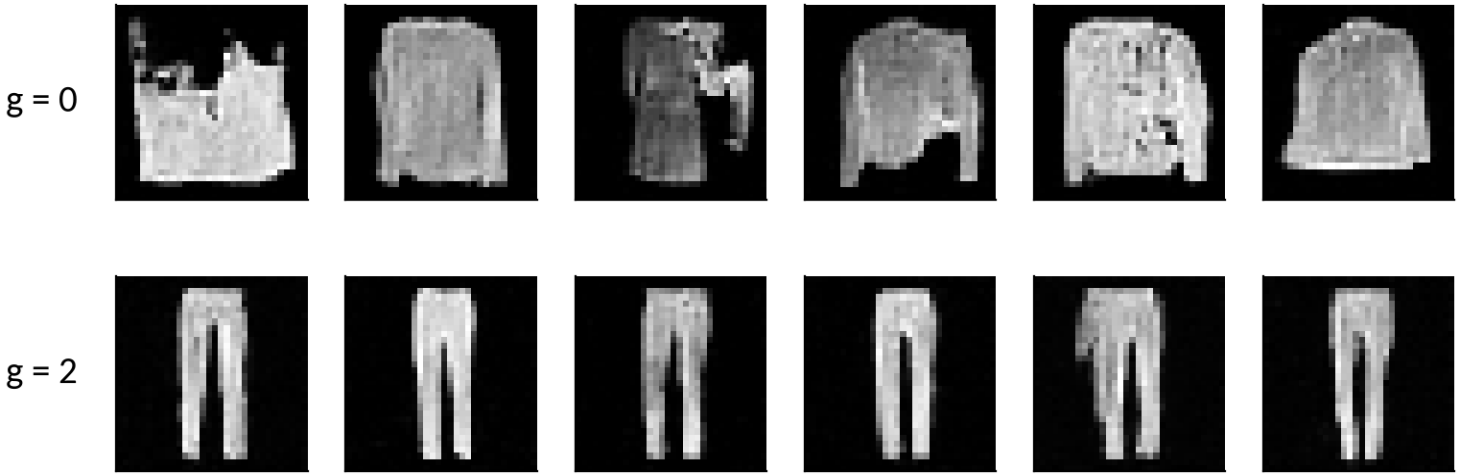
Source : Ho et Salimans, Classifier-Free Diffusion Guidance, 2022.

La force de la condition

Pourquoi : le réglage de l'étape 5, le guidance scale des générateurs d'images.

On prédit le bruit **sans** et **avec** la condition, puis on **amplifie l'écart** d'un facteur g. Avec g = 0, la condition est ignorée.

$$\text{bruit retenu} = \text{bruit sans condition} + g \times (\text{bruit avec condition} - \text{bruit sans condition})$$



U-Net fourni au TD, condition « pantalon », même bruit de départ pour les deux lignes.

- **g = 0** : des vêtements au hasard, la condition est ignorée
- **g = 2** : six pantalons
- Et au-delà ? C'est une question de l'**étape 5**

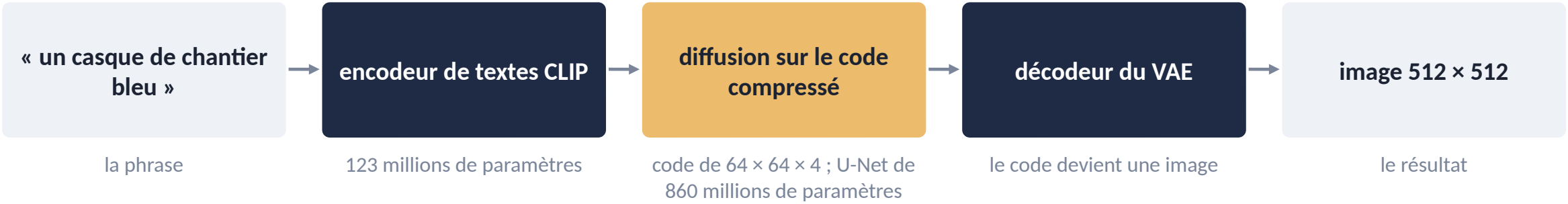
Dans l'article de Ho et Salimans, le même réglage s'écrit w = g − 1.

Source : U-Net fourni au TD (Fashion-MNIST) ; même bruit de départ pour les deux lignes ; calculé pour le cours.

Stable Diffusion : la diffusion dans un espace compressé

Pourquoi : relier le petit modèle du TD aux générateurs d'images actuels.

Stable Diffusion débruite **un code compressé** par un VAE, pas l'image : 48 fois moins de nombres. La condition est **une phrase encodée par CLIP**.



	Le modèle du TD	Stable Diffusion
ce que l'on débruite	une image 32 × 32	un code 64 × 64 × 4, décodé en image 512 × 512
condition	une classe parmi 10	une phrase, encodée par CLIP
U-Net	0,25 million de paramètres	860 millions de paramètres

Source : Rombach et al., High-Resolution Image Synthesis with Latent Diffusion Models, CVPR 2022 ; tailles : dépôt CompVis/stable-diffusion.

La diffusion conditionnée : à retenir

1. Le réseau reçoit une **condition** en plus : une classe au TD, une phrase encodée par CLIP dans Stable Diffusion.

2. **Force de la condition g** : on prédit le bruit avec et sans la condition, et on amplifie l'écart. g = 0 ignore la condition.

3. **Stable Diffusion** débruite un **code compressé** par un VAE (64 × 64 × 4 nombres au lieu de 512 × 512 × 3) : c'est ce qui la rend abordable.

Pour approfondir : Ho et Salimans, 2022 : Classifier-Free Diffusion Guidance (arXiv) ;
Rombach et al., 2022 : l'article de la diffusion latente (arXiv)

TD · étape 8

La brique projet : les défauts rares

Pourquoi : le volet défauts rares du projet, à l'étape 8.

Un type de défaut de tôle dont on n'a qu'**une quinzaine d'images** : deux pistes, toutes deux tirées de la matinée.

Piste A · retrouver

montrer à l'opérateur les défauts les plus proches de l'historique

embeddings CLIP, puis DINOv2 ; similarité cosinus

parties 1 à 4

Piste B · générer

créer des images synthétiques de la classe rare, puis réentraîner le classifieur

un petit modèle de diffusion ; mesure sur de vraies images

parties 5 à 7



Le groupe choisit lui-même la classe rare dans NEU et la réduit à **15 images**. Volet au choix du projet : voir **projet/SUJET.md**.

Source : Song, Yan et al., NEU surface defect database (NEU-DET) ; Oquab et al., DINOv2, 2023.

Synthèse

Pourquoi : les réponses aux trois questions, à garder en tête pour le TD.

1. **Classer sans exemple** : un foundation model (CLIP) projette images et textes dans un même espace ; une phrase par classe, la similarité cosinus, la température, puis un softmax.

2. **Chercher en décrivant** : le corpus est encodé une fois ; chaque requête est un produit matriciel et un tri. On évalue par la précision à k.

3. **Générer et choisir** : la diffusion apprend à retrouver le bruit, puis débruite un bruit pur pas à pas ; une condition, amplifiée par g, choisit ce qu'on génère.

4. **Mesurer avant de conclure** : chaque approche face à une baseline classique, les réglages (les phrases) choisis sur une partie des données, mesurés sur une autre.

Déroulé du TD 4

Pourquoi : chaque étape du TD applique une partie du cours.

Étape 2

CLIP : classer les vignettes sans aucun exemple, puis avec quelques exemples

parties 1 à 3 · TODO 1 et 2

Étape 3

CLIP : un moteur de recherche d'images par le texte

partie 4 · TODO 3 et 4

Étape 4

Diffusion sur des points du plan

partie 6 · TODO 5 et 6

Étape 5

Diffusion sur des images, avec une condition

partie 7

Étape 6

Fiche de décision ingénieur

synthèse

Étape 7

Bilan : que change un foundation model ?

parties 2 et 3, TD 2

Étape 8

Brique projet : les défauts rares (volet au choix)

toute la matinée



Étapes 0 et 1 : vérifier l'installation et revoir le cours en bref. Tout le TD tourne sur CPU.